

Capítulo 5: Redes neuronales

Sesión 3: Regularización en redes neuronales

Basado en C. M. Bishop, *Pattern Recognition and Machine Learning*

Sección 5.5: *Regularization in Neural Networks*

- 1 Sobreajuste
- 2 Weight decay y priors gaussianos
- 3 Priors gaussianos consistentes
- 4 Early stopping
- 5 Invariancias y tangent propagation
- 6 Entrenamiento con datos transformados
- 7 Regularización estructural: redes convolucionales
- 8 Soft weight sharing
- 9 Cierre

Idea central

ajustar los datos de entrenamiento $\neq$ generalizar bien.

- ▶ Entender por qué una red puede sobreajustar.
- ▶ Ver cómo la regularización modifica el criterio de entrenamiento.
- ▶ Conectar **weight decay** (regularizador cuadrático) con priors gaussianos.
- ▶ Interpretar **early stopping** como regularización implícita.
- ▶ Introducir invariancias, **tangent propagation**, datos transformados, CNNs y **soft weight sharing**.

Forward pass

$$\begin{aligned}a^{(1)} &= W^{(1)}x + b^{(1)}, & z^{(1)} &= h(a^{(1)}), \\a^{(2)} &= W^{(2)}z^{(1)} + b^{(2)}, & \hat{y} &= a^{(2)}.\end{aligned}$$

Regresión

$$E(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^N (y(x_n, \boldsymbol{w}) - t_n)^2.$$

Notación

Aquí los sesgos se escriben por separado. El vector $\boldsymbol{w}$ representa colectivamente todos los parámetros.

Observación

Una red suficientemente flexible puede producir

$$E_{\text{train}}(\mathbf{w}) \approx 0,$$

pero eso no implica

$$E_{\text{test}}(\mathbf{w}) \approx 0.$$

Idea

El entrenamiento elige una solución que ajusta datos. La regularización ayuda a elegir una solución **preferible** entre muchas posibles.

Para una red con una entrada, M unidades ocultas y salida lineal:

$$y(x, \mathbf{w}) = \sum_{j=1}^M w_j^{(2)} h\left(w_j^{(1)} x + b_j^{(1)}\right) + b^{(2)}.$$

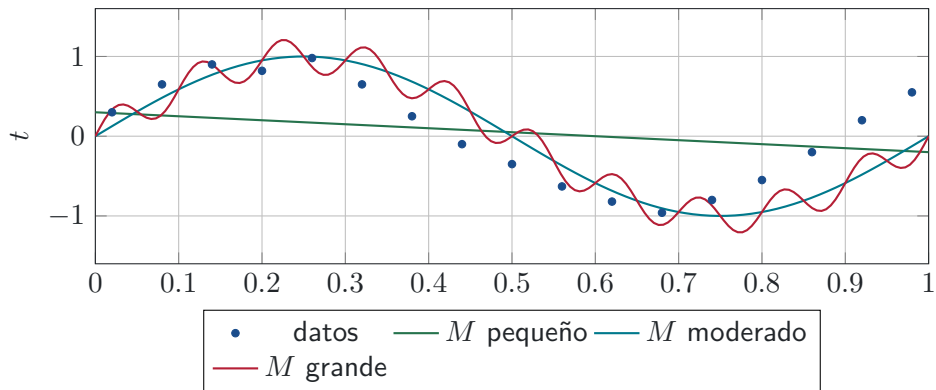
Idea

M pequeño $\Rightarrow$ subajuste, M grande $\Rightarrow$ riesgo de sobreajuste.

Advertencia

La complejidad efectiva del modelo no depende solamente de M .

Subajuste y sobreajuste



Fuente: Inspirado en Bishop, Fig. 5.9.

Entrenamiento

$$E_{\text{train}}(\mathbf{w}) = \frac{1}{2} \sum_{n \in \mathcal{D}_{\text{train}}} (y(x_n, \mathbf{w}) - t_n)^2.$$

Validación

$$E_{\text{val}}(\mathbf{w}) = \frac{1}{2} \sum_{n \in \mathcal{D}_{\text{val}}} (y(x_n, \mathbf{w}) - t_n)^2.$$

Advertencia

El conjunto de validación no se usa para actualizar pesos; se usa para escoger hiperparámetros, arquitectura o momento de parada (early stopping).

No depende de un solo número

complejidad efectiva: depende de M , λ , inicialización, tiempo, arquitectura, datos.

- ▶ M : número de unidades ocultas.
- ▶ λ : fuerza de regularización explícita.
- ▶ Tiempo de entrenamiento: puede limitar la capacidad explotada por el modelo.
- ▶ Arquitectura: puede imponer restricciones estructurales.

En lugar de minimizar solamente $E(\mathbf{w})$, minimizamos

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda\Omega(\mathbf{w}).$$

Interpretación

- ▶ $E(\mathbf{w})$: ajuste a datos.
- ▶ $\Omega(\mathbf{w})$: penalización o preferencia sobre parámetros/funciones.
- ▶ $\lambda \geq 0$: balance entre ajuste y regularización.

Regularizador cuadrático

$$\Omega(\mathbf{w}) = \frac{1}{2} \mathbf{w}^\top \mathbf{w}.$$

Entonces

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w} = E(\mathbf{w}) + \frac{\lambda}{2} \sum_i w_i^2.$$

Interpretación

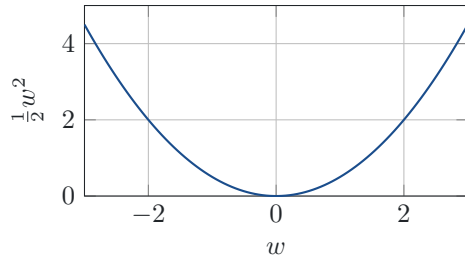
Weight decay favorece soluciones con pesos pequeños.

¿Qué penaliza weight decay?

Penalización

$$\Omega(w) = \frac{1}{2}w^2.$$

Un peso grande aporta mucho más al error que un peso pequeño.



Partimos de

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}.$$

Como

$$\nabla_{\mathbf{w}} \left(\frac{1}{2} \mathbf{w}^T \mathbf{w} \right) = \mathbf{w},$$

obtenemos

$$\nabla_{\mathbf{w}} \tilde{E}(\mathbf{w}) = \nabla_{\mathbf{w}} E(\mathbf{w}) + \lambda \mathbf{w}.$$

Cambio clave vs código de la sesión anterior

Solo hay que sumar $\lambda \mathbf{w}$ al gradiente ya calculado por backpropagation.

Para cada parámetro w_i :

$$\frac{\partial \tilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda w_i.$$

En notación de capas

$$dW_{\text{reg}}^{(1)} = dW^{(1)} + \lambda W^{(1)}, \quad dW_{\text{reg}}^{(2)} = dW^{(2)} + \lambda W^{(2)}.$$

Sesgos

En la práctica podemos decidir si regularizar o no los sesgos. Bishop explica que pesos y sesgos no siempre deben tratarse igual.

Descenso por gradiente:

$$w_i^{(\tau+1)} = w_i^{(\tau)} - \eta \left(\frac{\partial E}{\partial w_i} + \lambda w_i^{(\tau)} \right).$$

Reordenando:

$$w_i^{(\tau+1)} = (1 - \eta\lambda)w_i^{(\tau)} - \eta \frac{\partial E}{\partial w_i}.$$

Interpretación

El factor $(1 - \eta\lambda)$ contrae el peso en cada iteración: de ahí el nombre **decay**.

Supongamos un prior gaussiano sobre los pesos:

$$p(\mathbf{w} \mid \alpha) = \mathcal{N}(\mathbf{w} \mid 0, \alpha^{-1} I).$$

Entonces

$$p(\mathbf{w} \mid \alpha) \propto \exp\left(-\frac{\alpha}{2} \mathbf{w}^\top \mathbf{w}\right),$$

y por tanto

$$-\log p(\mathbf{w} \mid \alpha) = \frac{\alpha}{2} \mathbf{w}^\top \mathbf{w} + \text{constante}.$$

Por Bayes:

$$p(\mathbf{w} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \mathbf{w})p(\mathbf{w}).$$

Maximizar posterior equivale a minimizar

$$-\log p(\mathcal{D} \mid \mathbf{w}) - \log p(\mathbf{w}).$$

Si

$$-\log p(\mathcal{D} \mid \mathbf{w}) = E(\mathbf{w}),$$

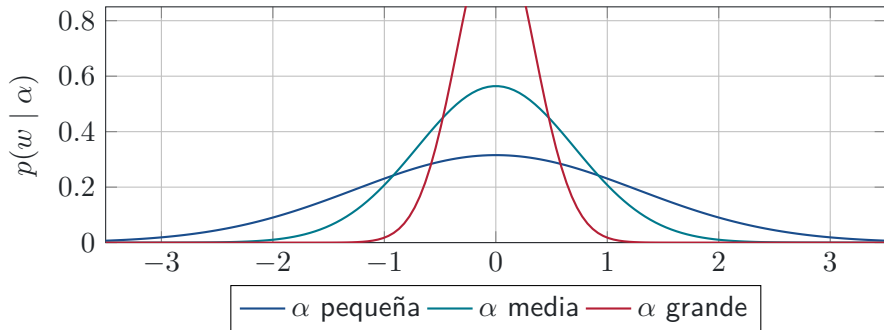
obtenemos

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \frac{\alpha}{2} \mathbf{w}^\top \mathbf{w}.$$

Correspondencia

En este contexto, λ juega el papel de la precisión α del prior.

Visualización del prior gaussiano



Interpretación

$$-\log p(w | \alpha) = \frac{\alpha}{2} w^2 + cte.$$

α grande concentra la densidad cerca de $w = 0$ y penaliza más los pesos alejados de cero.

Weight decay simple trata igual todos los parámetros:

$$W^{(1)}, \quad b^{(1)}, \quad W^{(2)}, \quad b^{(2)}.$$

Problema

Pesos y sesgos no se transforman igual cuando cambiamos la escala o el origen de los datos.

Idea

Un regularizador no debería favorecer arbitrariamente una solución solo porque medimos los datos en otras unidades.

Consideremos una primera capa

$$z_j = h \left(\sum_i w_{ji} x_i + w_{j0} \right).$$

Transformamos las entradas como

$$\tilde{x}_i = ax_i + b.$$

Queremos que la red pueda representar la misma función después del cambio de variables.

Pregunta

¿Cómo deben cambiar los pesos y sesgos de primera capa?

La activación original de la unidad oculta j es

$$a_j = \sum_i w_{ji}x_i + w_{j0}.$$

Si ahora alimentamos la red con variables transformadas

$$\tilde{x}_i = ax_i + b,$$

buscamos nuevos parámetros $\tilde{w}_{ji}$ y $\tilde{w}_{j0}$ tales que

$$\tilde{a}_j = \sum_i \tilde{w}_{ji}\tilde{x}_i + \tilde{w}_{j0} = \sum_i w_{ji}x_i + w_{j0} = a_j.$$

Idea

Como queremos la misma activación para todo x , igualamos coeficientes de x_i y el término constante.

Sustituyendo $\tilde{x}_i = ax_i + b$ en la activación transformada:

$$\tilde{a}_j = \sum_i \tilde{w}_{ji}(ax_i + b) + \tilde{w}_{j0}.$$

Separando términos que multiplican a x_i y términos constantes:

$$\tilde{a}_j = \sum_i (a\tilde{w}_{ji})x_i + \left(b \sum_i \tilde{w}_{ji} + \tilde{w}_{j0} \right).$$

Para que $\tilde{a}_j = a_j$ para todo x , necesitamos

$$a\tilde{w}_{ji} = w_{ji}, \quad b \sum_i \tilde{w}_{ji} + \tilde{w}_{j0} = w_{j0}.$$

Por tanto,

$$\tilde{w}_{ji} = \frac{1}{a}w_{ji}, \quad \tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji}.$$

Transformación de pesos de primera capa

Para preservar la misma activación bajo

$$\tilde{x}_i = ax_i + b,$$

podemos usar

$$w_{ji} \mapsto \tilde{w}_{ji} = \frac{1}{a}w_{ji},$$

$$w_{j0} \mapsto \tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji}.$$

Interpretación

El cambio de escala a reescala los pesos. La traslación b modifica el sesgo y lo mezcla con la suma de los pesos.

Consecuencia

Penalizar sesgos y pesos de la misma manera puede depender de la escala u origen usados para medir los datos.

Partimos de la segunda capa

$$y_k = \sum_j w_{kj} z_j + w_{k0}.$$

Si queremos transformar la variable de salida como

$$\tilde{y}_k = cy_k + d,$$

entonces sustituimos la expresión de y_k :

$$\tilde{y}_k = c \left(\sum_j w_{kj} z_j + w_{k0} \right) + d = \sum_j (cw_{kj}) z_j + (cw_{k0} + d).$$

Conclusión

Para preservar la misma relación funcional, basta tomar

$$w_{kj} \mapsto \tilde{w}_{kj} = cw_{kj}, \quad w_{k0} \mapsto \tilde{w}_{k0} = cw_{k0} + d.$$

Por qué weight decay simple no es consistente

El regularizador simple penaliza todos los parámetros por igual:

$$\Omega_{\text{simple}} = \frac{\lambda}{2} \left(\sum_{j,i} w_{ji}^2 + \sum_j w_{j0}^2 + \sum_{k,j} w_{kj}^2 + \sum_k w_{k0}^2 \right).$$

Pero bajo una traslación de entrada, el sesgo cambia como

$$\tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji}.$$

Entonces

$$\sum_j \tilde{w}_{j0}^2 = \sum_j \left(w_{j0} - \frac{b}{a} \sum_i w_{ji} \right)^2,$$

que no es simplemente una constante por $\sum_j w_{j0}^2$.

Consecuencia

Dos redes que representan la misma función, pero usan unidades u origen distintos para los datos, pueden recibir penalizaciones distintas.

Inconsistencia bajo transformación de salida

Bajo

$$\tilde{y}_k = cy_k + d,$$

los parámetros de salida se transforman como

$$\tilde{w}_{kj} = cw_{kj}, \quad \tilde{w}_{k0} = cw_{k0} + d.$$

Por tanto, la penalización de los sesgos de salida se convierte en

$$\sum_k \tilde{w}_{k0}^2 = \sum_k (cw_{k0} + d)^2.$$

Interpretación

El sesgo no solo cambia de escala: también se desplaza por d .

Advertencia

Por eso penalizar pesos y sesgos de manera igual puede depender de la escala y del origen elegidos para las variables.

En vez de usar un único

$$\frac{\lambda}{2} \mathbf{w}^\top \mathbf{w},$$

Bishop propone separar pesos por capas:

$$\Omega_{\text{grupo}} = \frac{\lambda_1}{2} \sum_{w \in W_1} w^2 + \frac{\lambda_2}{2} \sum_{w \in W_2} w^2.$$

Definición

W_1 es el conjunto de pesos de la primera capa y W_2 el conjunto de pesos de la segunda capa.

Importante

En esta forma, los **sesgos se excluyen** de las sumatorias. Si se desean regularizar, conviene asignarles hiperparámetros separados.

Objetivo

Mostrar que el término de primera capa puede conservarse si el hiperparámetro se reescala junto con el cambio de unidades.

Bajo $\tilde{x}_i = ax_i + b$:

$$\tilde{w}_{ji} = \frac{1}{a} w_{ji}.$$

Entonces

$$\sum_{w \in \tilde{W}_1} w^2 = \sum_{j,i} \left(\frac{1}{a} w_{ji} \right)^2 = \frac{1}{a^2} \sum_{w \in W_1} w^2.$$

Si λ_1 es el coeficiente de la penalización cuadrática y

$$\tilde{\lambda}_1 = a^2 \lambda_1,$$

se obtiene

$$\frac{\tilde{\lambda}_1}{2} \sum_{w \in \tilde{W}_1} w^2 = \frac{a^2 \lambda_1}{2} \frac{1}{a^2} \sum_{w \in W_1} w^2 = \frac{\lambda_1}{2} \sum_{w \in W_1} w^2.$$

Objetivo

Mostrar que el término de segunda capa también se conserva bajo un reescalamiento adecuado.

Bajo $\tilde{y}_k = cy_k + d$:

$$\tilde{w}_{kj} = cw_{kj}.$$

Entonces

$$\sum_{w \in \tilde{W}_2} w^2 = \sum_{k,j} (cw_{kj})^2 = c^2 \sum_{w \in W_2} w^2.$$

Si

$$\tilde{\lambda}_2 = c^{-2} \lambda_2,$$

entonces

$$\frac{\tilde{\lambda}_2}{2} \sum_{w \in \tilde{W}_2} w^2 = \frac{c^{-2} \lambda_2}{2} c^2 \sum_{w \in W_2} w^2 = \frac{\lambda_2}{2} \sum_{w \in W_2} w^2.$$

Nota

Aquí λ_1, λ_2 son los coeficientes que multiplican las sumas de cuadrados en la función objetivo.

Propiedad buscada

Si dos redes representan la misma función después de transformar linealmente entradas o salidas, el regularizador no debería preferir una de ellas de forma arbitraria.

- ▶ El regularizador simple mezcla pesos y sesgos en una sola penalización.
- ▶ Los sesgos cambian con traslaciones: b en la entrada y d en la salida.
- ▶ El regularizador por grupos evita ese problema al separar escalas y excluir sesgos de las sumatorias principales.
- ▶ Los hiperparámetros λ_1 y λ_2 absorben los cambios de escala de las variables.

Idea

La regularización debe penalizar propiedades del modelo solamente, no elementos arbitrarios como la unidad de medida u origen/desplazamiento de los datos.

Podemos dividir los pesos en grupos arbitrarios:

$$W_1, W_2, \dots, W_K.$$

Por ejemplo:

pesos por capa, pesos asociados a cada entrada, sesgos por separado.

El regularizador por grupos

$$\Omega(\mathbf{w}) = \frac{1}{2} \sum_k \alpha_k \|\mathbf{w}\|_k^2$$

corresponde al prior

$$p(\mathbf{w}) \propto \exp \left(-\frac{1}{2} \sum_k \alpha_k \|\mathbf{w}\|_k^2 \right), \quad \|\mathbf{w}\|_k^2 = \sum_{j \in W_k} w_j^2.$$

Justificación

En estimación MAP minimizamos

$$-\log p(\mathcal{D} \mid \mathbf{w}) - \log p(\mathbf{w}).$$

Como

$$-\log p(\mathbf{w}) = \frac{1}{2} \sum_k \alpha_k \|\mathbf{w}\|_k^2 + \text{cte.},$$

el negativo del log-prior produce el regularizador por grupos.

Interpretación

Cada grupo tiene su propia precisión α_k , es decir, su propia fuerza de regularización.

Durante entrenamiento:

$$E_{\text{train}}(\tau) \downarrow,$$

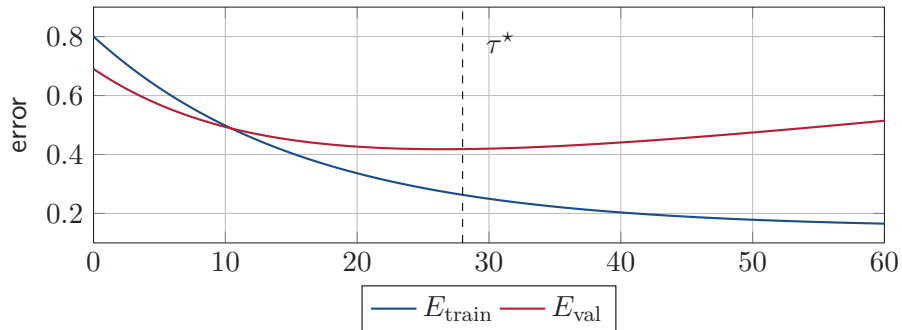
pero el error de validación suele comportarse como

$$E_{\text{val}}(\tau) : \text{baja al inicio y luego sube.}$$

Regla:

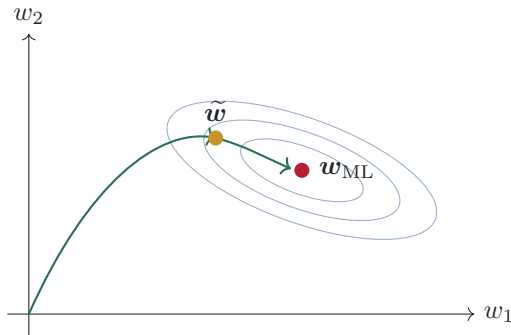
$$\tau^* = \arg \min_{\tau} E_{\text{val}}(\tau), \quad \mathbf{w}_{\text{final}} = \mathbf{w}^{(\tau^*)}.$$

Curvas de entrenamiento y validación



¿Por qué early stopping regulariza?

Early stopping no agrega una penalización explícita a $E(\mathbf{w})$, pero limita la región del espacio de pesos que se alcanza durante la optimización.



Fuente: Inspirado en Bishop, Fig. 5.13.

Idea

pocas iteraciones $\Rightarrow$ la red no explota toda su capacidad.

muchas iteraciones $\Rightarrow$ la red puede ajustar ruido.

Supongamos que, cerca del mínimo no regularizado $\mathbf{w}^*$, el error se aproxima por

$$E(\mathbf{w}) = E_0 + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top H(\mathbf{w} - \mathbf{w}^*).$$

Notación

- ▶ $\mathbf{w}$: vector con todos los parámetros de la red.
- ▶ $\mathbf{w}^*$: mínimo de la aproximación cuadrática.
- ▶ H : matriz Hessiana del error evaluada cerca de $\mathbf{w}^*$.
- ▶ u_j : eigenvector de H y η_j : eigenvalor asociado,

$$Hu_j = \eta_j u_j.$$

- ▶ $w_j^{(\tau)}$ y w_j^* : coordenadas de $\mathbf{w}^{(\tau)}$ y $\mathbf{w}^*$ en la dirección u_j .

Si nos movemos desde el mínimo en una sola dirección u_j ,

$$\mathbf{w} = \mathbf{w}^* + \alpha u_j,$$

entonces

$$E(\mathbf{w}) = E_0 + \frac{1}{2}\alpha^2 u_j^\top H u_j.$$

Como $Hu_j = \eta_j u_j$ y $u_j^\top u_j = 1$,

$$E(\mathbf{w}^* + \alpha u_j) = E_0 + \frac{1}{2}\eta_j \alpha^2.$$

Interpretación

El eigenvalor η_j mide la **curvatura** del error en la dirección u_j : si η_j es grande, el error crece más rápido al moverse en esa dirección.

Para la aproximación cuadrática,

$$\nabla E(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*).$$

Descenso por gradiente con tasa ρ da

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \rho H(\mathbf{w}^{(\tau)} - \mathbf{w}^*).$$

Escribimos

$$\mathbf{w}^{(\tau)} = \sum_j w_j^{(\tau)} u_j, \quad \mathbf{w}^* = \sum_j w_j^* u_j.$$

Usando $Hu_j = \eta_j u_j$,

$$w_j^{(\tau+1)} = w_j^{(\tau)} - \rho \eta_j \left(w_j^{(\tau)} - w_j^* \right).$$

Definimos el error de coordenada

$$r_j^{(\tau)} = w_j^{(\tau)} - w_j^*.$$

De la recurrencia anterior,

$$r_j^{(\tau+1)} = (1 - \rho\eta_j)r_j^{(\tau)}.$$

Iterando,

$$r_j^{(\tau)} = (1 - \rho\eta_j)^\tau r_j^{(0)}.$$

Si el entrenamiento empieza en el origen, $w_j^{(0)} = 0$, entonces $r_j^{(0)} = -w_j^*$ y

$$w_j^{(\tau)} = [1 - (1 - \rho\eta_j)^\tau] w_j^*.$$

Para $\rho\eta_j$ pequeño,

$$(1 - \rho\eta_j)^\tau \approx e^{-\rho\eta_j\tau}.$$

Entonces

$$w_j^{(\tau)} \approx [1 - e^{-\rho\eta_j\tau}] w_j^*.$$

Idea

Las direcciones con mayor curvatura tienen η_j grande, por lo que $e^{-\rho\eta_j\tau}$ decae más rápido y $w_j^{(\tau)}$ se acerca antes a w_j^* .

Idea

Detener temprano permite aprender primero las direcciones dominantes antes de ajustar variaciones finas o ruido.

En los ejemplos anteriores:

regularización $\approx$ preferir pesos pequeños.

Pero la regularización también puede ser:

regularización $\approx$ preferir salidas estables bajo transformaciones (invarianzas y equivarianzas).

Motivación

En reconocimiento de patrones muchas etiquetas no cambian ante transformaciones pequeñas de la entrada. Ejemplos: dígitos desplazados en una imagen, señales con pequeños desfases.

Sea una transformación parametrizada por ξ :

$$s(x, \xi), \quad s(x, 0) = x.$$

Una red es invariante si

$$y(s(x, \xi), \mathbf{w}) = y(x, \mathbf{w}).$$

Ejemplos de transformaciones de interés

Traslación pequeña, rotación pequeña, cambio de escala, deformación suave.

1. Aumentar el conjunto con datos transformados.
2. Añadir un regularizador que penalice cambios de salida.
3. Extraer características invariantes en preprocesamiento.
4. Construir la invariancia en la arquitectura.

Ejemplos

Tangent propagation (punto 2), redes convolucionales (punto 4). Los cuales se revisarán a continuación.

Invariancia

$$y(s(x, \xi)) = y(x).$$

La salida no cambia.

Ejemplo

En CNNs, las representaciones intermedias suelen ser equivariantes a traslación; la decisión final busca invariancia aproximada.

Equivarianza

$$f(s(x, \xi)) = s'(f(x), \xi).$$

La representación cambia de forma predecible.

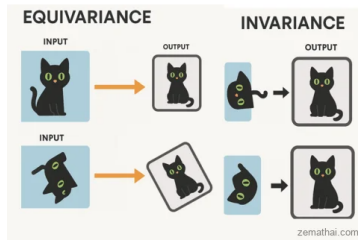
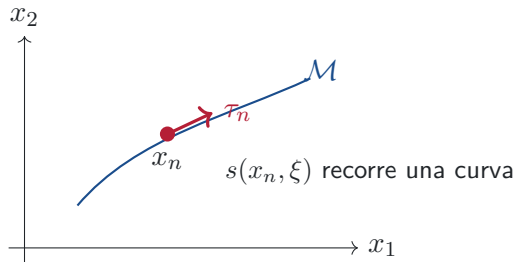


Figura 1: Invariance vs equivariance. Source: <https://zeromathai.com/es/equivariance-vs-invariance-es/>

Consideremos una transformación continua, parametrizada por ξ , sobre un vector particular x_n , que barre una variedad $\mathcal{M}$ (1-variedad o curva cuando $D = 2$) en el espacio de entrada D -dimensional; localmente se aproxima por un vector tangente τ_n . El vector que resulta de actuar sobre x_n por esta transformación es denotado por $s(x_n, \xi)$.



Fuente: Inspirado en Bishop, Fig. 5.15.

El vector tangente se define como

$$\tau_n = \left. \frac{\partial s(x_n, \xi)}{\partial \xi} \right|_{\xi=0}.$$

Interpretación

τ_n = dirección instantánea en la que se mueve x_n bajo la transformación.

Ejemplo

Si $s(x, \xi) = x + \xi v$, entonces $\tau = v$.

Por regla de la cadena:

$$\left. \frac{\partial y_k}{\partial \xi} \right|_{\xi=0} = \sum_{i=1}^D \frac{\partial y_k}{\partial x_i} \left. \frac{\partial x_i}{\partial \xi} \right|_{\xi=0}.$$

Si

$$J_{ki} = \frac{\partial y_k}{\partial x_i}, \quad \tau_i = \left. \frac{\partial x_i}{\partial \xi} \right|_{\xi=0},$$

entonces

$$\left. \frac{\partial y_k}{\partial \xi} \right|_{\xi=0} = \sum_{i=1}^D J_{ki} \tau_i.$$

Queremos que la salida cambie poco en la dirección de la transformación:

$$\Omega = \frac{1}{2} \sum_n \sum_k \left(\left. \frac{\partial y_{nk}}{\partial \xi} \right|_{\xi=0} \right)^2.$$

Sustituyendo:

$$\Omega = \frac{1}{2} \sum_n \sum_k \left(\sum_i J_{nki} \tau_{ni} \right)^2.$$

El error total es

$$\tilde{E} = E + \lambda \Omega.$$

Idea

La penalización es cero si la red es localmente invariante a la transformación alrededor de cada dato.

Usando Taylor:

$$y(x_n + \epsilon \tau_n) \approx y(x_n) + \epsilon \nabla_x y(x_n)^\top \tau_n.$$

Si queremos

$$y(x_n + \epsilon \tau_n) \approx y(x_n),$$

entonces queremos que

$$\nabla_x y(x_n)^\top \tau_n \approx 0.$$

Idea

Tangent propagation penaliza sensibilidad local en direcciones que deberían ser irrelevantes.

Cuando no tenemos una fórmula analítica para τ_n , podemos aproximarlo por diferencias finitas:

$$\tau_n \approx \frac{s(x_n, \epsilon) - x_n}{\epsilon}.$$

Interpretación

Transformamos ligeramente el dato, restamos el original y dividimos por el tamaño de la transformación.

Advertencia

Esta aproximación representa una invariancia local, no necesariamente una invariancia global.

Otra forma de introducir invarianzas en el modelo es en lugar de entrenar solo con

$$(x_n, t_n),$$

entrenamos también con

$$(s(x_n, \xi), t_n).$$

Idea

Si la etiqueta no cambia bajo la transformación, podemos enseñar esa invariancia mediante ejemplos transformados.

Hasta ahora usamos un error discreto sobre datos observados:

$$E_N = \frac{1}{2N} \sum_{n=1}^N \{y(x_n) - t_n\}^2.$$

Si los pares (x_n, t_n) son muestras de una distribución conjunta $p(x, t)$, el promedio empírico se aproxima por una esperanza:

$$\frac{1}{N} \sum_{n=1}^N f(x_n, t_n) \approx \mathbb{E}_{p(x,t)}[f(x, t)].$$

Como $p(x, t) = p(t \mid x)p(x)$, la esperanza se escribe como

$$E = \frac{1}{2} \int \int \{y(x) - t\}^2 p(t \mid x) p(x) \, dx \, dt.$$

Lectura

La suma sobre datos se reemplaza por una integral sobre todos los posibles valores de x y t , ponderados por su probabilidad.

Con data augmentation, cada dato puede generar copias transformadas:

$$(x_n, t_n) \mapsto (s(x_n, \xi_r), t_n).$$

Un error empírico promedio tendría la forma

$$\tilde{E}_{N,R} = \frac{1}{2NR} \sum_{n=1}^N \sum_{r=1}^R \{y(s(x_n, \xi_r)) - t_n\}^2.$$

En el límite poblacional, la suma sobre datos se vuelve una integral sobre x, t y la suma sobre transformaciones se vuelve una integral sobre ξ :

$$\tilde{E} = \frac{1}{2} \int \int \int \{y(s(x, \xi)) - t\}^2 p(t | x) p(x) p(\xi) dx dt d\xi.$$

Hipótesis

Bishop considera transformaciones pequeñas con $\mathbb{E}[\xi] = 0$ y $\mathbb{E}[\xi^2] = \lambda$.

Para transformaciones pequeñas:

$$s(x, \xi) = s(x, 0) + \xi \left. \frac{\partial s(x, \xi)}{\partial \xi} \right|_{\xi=0} + \frac{\xi^2}{2} \left. \frac{\partial^2 s(x, \xi)}{\partial \xi^2} \right|_{\xi=0} + O(\xi^3).$$

Como $s(x, 0) = x$:

$$s(x, \xi) = x + \xi \tau + \frac{1}{2} \xi^2 \tau' + O(\xi^3).$$

Sustituyendo la expansión de $s(x, \xi)$ en y :

$$y(s(x, \xi)) = y(x) + \xi \tau^T \nabla y(x) + \frac{\xi^2}{2} [(\tau')^T \nabla y(x) + \tau^T \nabla \nabla y(x) \tau] + O(\xi^3).$$

Interpretación

El primer término nuevo es la derivada direccional $\tau^T \nabla y(x)$.

Al sustituir la expansión de Taylor en el error y usar

$$\mathbb{E}[\xi] = 0, \quad \mathbb{E}[\xi^2] = \lambda,$$

los términos lineales en ξ desaparecen. Omitiendo términos de orden $O(\xi^3)$:

$$\tilde{E} \approx E + \lambda \Omega.$$

El término que queda penaliza la derivada direccional de la salida:

$$\Omega = \frac{1}{2} \int (\tau^\top \nabla y(x))^2 p(x) \, dx.$$

Como

$$\tau^\top \nabla y(x) = \left. \frac{\partial y(s(x, \xi))}{\partial \xi} \right|_{\xi=0},$$

esta es la versión poblacional del regularizador de tangent propagation.

Lectura

Entrenar con pequeñas copias transformadas equivale, localmente, a penalizar cambios de la salida en la dirección tangente de la transformación.

Si la transformación es simplemente añadir ruido pequeño:

$$x \mapsto x + \xi,$$

entonces el regularizador resultante toma la forma

$$\Omega = \frac{1}{2} \int \|\nabla y(x)\|^2 p(x) dx.$$

Nombre

Este tipo de penalización se conoce como regularización de Tikhonov.

Otra forma de regularizar no consiste en cambiar $E(\boldsymbol{w})$, sino en cambiar la arquitectura del modelo para introducir invariantes. Consideremos la tarea de clasificación de dígitos escritos a mano. Las redes neuronales como las vistas anteriormente ignorarían el hecho de que píxeles cercanos están más fuertemente correlacionados que los píxeles lejanos, por lo que surge la necesidad de extraer representaciones locales que solo dependan de subconjuntos pequeños de una imagen, luego, dichas representaciones locales intermedias pueden ser combinadas en capas posteriores para detectar características de alto nivel

Idea

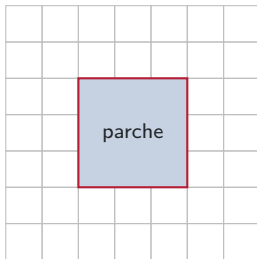
La arquitectura restringe qué funciones puede representar la red y qué parámetros se comparten.

CNNs según Bishop

Las redes convolucionales incorporan tres mecanismos:

campos receptivos locales, compartición de pesos, submuestreo.

Bishop	Término equivalente o significado
Receptive field	<i>Campo receptivo local</i>
Weight sharing	<i>Compartición de pesos</i> : Idea de que el mismo filtro/kernel (con ciertos parámetros o <i>pesos</i>) es aplicado en varias posiciones
Units in a plane	<i>Feature map</i> : mapa de activaciones (unidades) resultado de aplicar un filtro
Subsampling	Reducción espacial, típicamente <i>pooling</i> o <i>submuestreo</i>
Stride	Tamaño de paso usado en la aplicación del filtro (convolución)



Definición

El **campo receptivo** es la región de la entrada (ej. imagen) que considera el filtro en determinada posición

Para una primera capa convolucional, el resultado (o *activación*) de aplicar un filtro $R \times S$, en posición (u, v) depende solo del parche y sirve como entrada para una sola neurona en particular.

$$\{X_{u+r,v+s} : 0 \leq r < R, 0 \leq s < S\}.$$

Consecuencia

La red no conecta cada neurona con todos los píxeles; primero extrae información local.

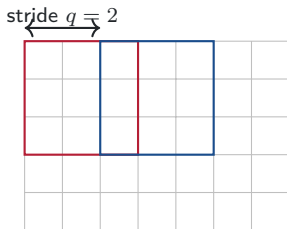
Sea una imagen de entrada $X \in \mathbb{R}^{H \times W}$ y un filtro $K \in \mathbb{R}^{R \times S}$. Sin padding y con stride q , una *activación* se calcula como

$$A_{u,v} = b + \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} K_{r,s} X_{uq+r, vq+s}.$$

Notación

- ▶ $K_{r,s}$: peso del filtro en la posición local (r, s) .
- ▶ b : sesgo compartido del filtro.
- ▶ (u, v) : posición espacial en el mapa de salida.
- ▶ q : *stride*, es decir, tamaño del salto del filtro.

Cada posición (u, v) corresponde a una ubicación distinta del filtro sobre la imagen.



Para convolución válida sin padding:

$$H_{\text{out}} = \left\lfloor \frac{H - R}{q} \right\rfloor + 1, \quad W_{\text{out}} = \left\lfloor \frac{W - S}{q} \right\rfloor + 1.$$

Interpretación

El stride controla cuántas posiciones espaciales se calculan: stride mayor produce mapas más pequeños.

En la práctica se usan varios filtros. Para el filtro ℓ , la activación en la posición (u, v) queda como:

$$A_{u,v}^{(\ell)} = b^{(\ell)} + \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} K_{r,s}^{(\ell)} X_{uq+r, vq+s}.$$

Después de aplicar una no linealidad h :

$$Z_{u,v}^{(\ell)} = h\left(A_{u,v}^{(\ell)}\right).$$

Feature map

El arreglo $Z^{(\ell)}$ es el **feature map** producido por el filtro ℓ .

Idea

Cada filtro aprende a responder a una característica local distinta: bordes, orientaciones, texturas o patrones más abstractos en capas profundas.

Después de calcular $A_{u,v}^{(\ell)}$ se aplica una activación no lineal:

$$Z_{u,v}^{(\ell)} = h\left(A_{u,v}^{(\ell)}\right).$$

Una opción común es ReLU:

$$h(a) = \max(0, a).$$

Por qué es necesaria también en CNNs

Sin no linealidades, una composición de capas lineales sigue siendo una transformación lineal. La red profunda no ganaría expresividad esencial.

Interpretación

La convolución produce respuestas locales; la no linealidad permite combinarlas para representar patrones más complejos.

Para imágenes con varios canales, por ejemplo RGB,

$$X \in \mathbb{R}^{H \times W \times C}, \quad K^{(\ell)} \in \mathbb{R}^{R \times S \times C}.$$

La convolución suma también sobre canales:

$$A_{u,v}^{(\ell)} = b^{(\ell)} + \sum_{c=1}^C \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} K_{r,s,c}^{(\ell)} X_{uq+r, vq+s, c}.$$

Si hay L filtros, la salida Z de 1 sola capa convolucional tiene forma aproximada

$$H_{\text{out}} \times W_{\text{out}} \times L.$$

Interpretación

La profundidad de salida es el número de filtros, no el número de canales originales.

En una capa totalmente conectada (de una red neuronal MLP), cada posición podría tener sus propios pesos. En una convolución, el mismo filtro se reutiliza en todas las posiciones.

Parámetros por filtro

Para un filtro $R \times S \times C$:

$$\# \text{parámetros} = RSC + 1,$$

contando el sesgo.

Sin compartir pesos

Habría un conjunto distinto de pesos para cada posición (u, v) .

$$\# \text{parámetros} \propto H_{\text{out}} W_{\text{out}} RSC.$$

Conexión con regularización

Compartir pesos es una restricción fuerte: obliga a detectar la misma característica local en cualquier posición.

Si una imagen se desplaza, una convolución tiende a desplazar también su mapa de activaciones:

$$X \text{ desplazada} \implies Z^{(\ell)} \text{ desplazado.}$$

Equivarianza

La convolución no hace que la representación sea idéntica; hace que cambie de forma predecible con la transformación.

Invariancia aproximada

La invariancia se obtiene de manera progresiva al combinar convoluciones, no linealidades, pooling y capas finales de clasificación.

Bishop habla de **subsampling**. En la literatura moderna, una técnica común es **max pooling**. Para una ventana $R_p \times S_p$ y stride q_p :

$$P_{u,v}^{(\ell)} = \max_{0 \leq r < R_p, 0 \leq s < S_p} Z_{uq_p+r, vq_p+s}^{(\ell)}.$$

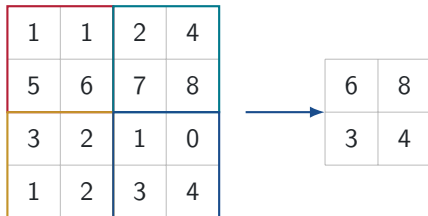
Interpretación

Max pooling conserva la respuesta más fuerte en una región local y reduce la resolución espacial.

Relación con subsampling

Esto corresponde a un caso particular de subsampling y permite pasar de un mapa grande a un mapa más pequeño, con menor sensibilidad a desplazamientos pequeños.

Ejemplo de max pooling



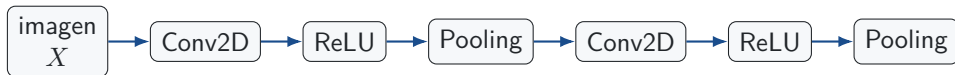
Con ventana 2×2 y stride 2, cada bloque se reemplaza por su máximo.

Efecto

- ▶ Reduce dimensión.
- ▶ Resume respuestas locales.
- ▶ Aporta invariancia local aproximada.

En la práctica se repite el patrón:

Conv2D $\longrightarrow$ activación no lineal $\longrightarrow$ pooling.



Interpretación jerárquica

Las primeras capas detectan patrones simples; capas posteriores combinan esos patrones para obtener características de más alto nivel.

Después de varias capas convolucionales y de pooling, obtenemos un tensor de características (en el sentido definido por <https://www.tensorflow.org/guide/tensor>):

$$P \in \mathbb{R}^{H' \times W' \times L}.$$

Para usar una red como las vistas antes, lo convertimos en un vector:

$$z = \text{vec}(P) \in \mathbb{R}^{H'W'L}.$$

Luego podemos aplicar una capa totalmente conectada:

$$a = Wz + b,$$

seguida de una salida adecuada, por ejemplo softmax para clasificación multiclase.

Idea

La parte convolucional aprende características; la parte densa combina esas características para tomar la decisión final.



Tres roles

- ▶ Convolución: extrae features locales usando filtros compartidos.
- ▶ Activación: introduce no linealidad, por ejemplo, con función ReLU.
- ▶ Pooling: reduce dimensión y aporta estabilidad local.

Nota

La regularización proviene de la estructura: campos locales, pesos compartidos y subsampling reducen la complejidad efectiva. Densa: se refiere a una red neuronal densamente conectada (MLP). Softmax: se usa como salida para clasificación multiclase.

Si un mismo parámetro w se usa en varias posiciones, su gradiente total suma todas las contribuciones:

$$\frac{\partial E}{\partial w} = \sum_{r \in \text{usos de } w} \frac{\partial E}{\partial w^{(r)}}.$$

Para un filtro $K_{r,s,c}^{(\ell)}$:

$$\frac{\partial E}{\partial K_{r,s,c}^{(\ell)}} = \sum_{u,v} \frac{\partial E}{\partial A_{u,v}^{(\ell)}} \frac{\partial A_{u,v}^{(\ell)}}{\partial K_{r,s,c}^{(\ell)}}.$$

Como

$$\frac{\partial A_{u,v}^{(\ell)}}{\partial K_{r,s,c}^{(\ell)}} = X_{uq+r, vq+s, c},$$

se acumula información de todas las posiciones donde se usó el filtro.

Como vimos para el caso de las CNN, una manera de regularizar es compartir los mismos valores para ciertos grupos de pesos del modelo y así reducir complejidad.

Esta idea puede ser reemplazada por una forma de regularización en la cual grupos de pesos tienen valores similares en lugar de exactamente iguales, conocida como *soft sharing*.

Hard sharing

$$w_i = w_j.$$

Igualdad exacta impuesta por la arquitectura. En una CNN estándar, cada filtro/kernel usa hard weight sharing.

Soft sharing

$$w_i \approx w_j.$$

Los pesos son alentados a agruparse, pero no obligados a ser iguales.

En soft weight sharing se supone que cada peso w fue generado por uno de varios grupos gaussianos. El coeficiente π_j representa la probabilidad a priori de que un peso pertenezca al grupo j , mientras que

$$\mathcal{N}(w \mid \mu_j, \sigma_j^2)$$

mide qué tan compatible es el valor observado de w con ese grupo.

Para cada peso:

$$p(w_i) = \sum_{j=1}^M \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2).$$

Para todos los pesos:

$$p(\mathbf{w}) = \prod_i p(w_i).$$

Interpretación

En vez de atraer todos los pesos hacia cero, permitimos varios centros aprendidos μ_j . M aquí es número de componentes de mezcla

Tomando el negativo del log-prior:

$$\Omega(\mathbf{w}) = - \sum_i \ln \left(\sum_{j=1}^M \pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) \right).$$

El error total queda

$$\tilde{E}(\mathbf{w}) = E(\mathbf{w}) + \lambda \Omega(\mathbf{w}).$$

Advertencia

La regularización depende ahora de parámetros de mezcla π_j, μ_j, σ_j que también se aprenden. Es recomendable usar métodos de tipo quasi-Newton para optimizar los parámetros de mezcla y el resto de pesos del modelo en conjunto, para evitar inestabilidad numérica en el entrenamiento.

Por Bayes, la probabilidad posterior de que w pertenezca al grupo j es proporcional a

$$\pi_j \mathcal{N}(w \mid \mu_j, \sigma_j^2).$$

Al normalizar sobre todos los grupos posibles se obtiene

$$\gamma_j(w) = \frac{\pi_j \mathcal{N}(w \mid \mu_j, \sigma_j^2)}{\sum_k \pi_k \mathcal{N}(w \mid \mu_k, \sigma_k^2)}.$$

Por eso $\gamma_j(w)$ puede interpretarse como la responsabilidad del grupo j sobre el peso w .

Interpretación

$\gamma_j(w_i)$ = grado de pertenencia del peso w_i al grupo j .

Importante

No es una asignación estricta: un peso puede pertenecer parcialmente a varios grupos.

Dado el error regularizado anterior,

$$\frac{\partial \tilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda \frac{\partial \Omega}{\partial w_i}.$$

Al derivar el término Ω respecto a w_i , solamente aparece el término asociado a ese peso:

$$\frac{\partial \Omega}{\partial w_i} = - \frac{\sum_j \pi_j \frac{\partial}{\partial w_i} \mathcal{N}(w_i \mid \mu_j, \sigma_j^2)}{\sum_k \pi_k \mathcal{N}(w_i \mid \mu_k, \sigma_k^2)}.$$

Como

$$\frac{\partial}{\partial w_i} \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = - \frac{w_i - \mu_j}{\sigma_j^2} \mathcal{N}(w_i \mid \mu_j, \sigma_j^2),$$

obtenemos

$$\frac{\partial \Omega}{\partial w_i} = \sum_j \frac{\pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2)}{\sum_k \pi_k \mathcal{N}(w_i \mid \mu_k, \sigma_k^2)} \frac{w_i - \mu_j}{\sigma_j^2}.$$

Usando la notación anterior de responsabilidad $\gamma_j(w_i)$ queda

$$\frac{\partial \tilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda \sum_j \gamma_j(w_i) \frac{w_i - \mu_j}{\sigma_j^2}.$$

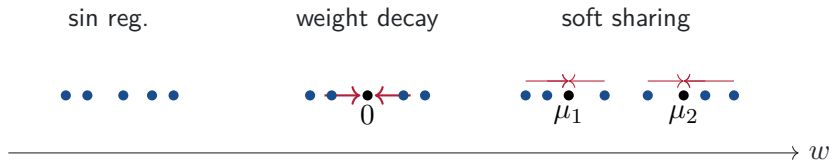
Interpretación

El término regularizador adicional actúa como una fuerza de atracción sobre cada peso w_i . Cada centro μ_j jala al peso con intensidad proporcional a dos factores:

$$\gamma_j(w_i) \quad \text{y} \quad \frac{1}{\sigma_j^2}.$$

Así, un grupo explica más al peso si tiene mayor responsabilidad $\gamma_j(w_i)$, y lo atrae con más fuerza si su varianza σ_j^2 es pequeña. Si w_i está por encima de μ_j , el gradiente positivo hace que el descenso lo reduzca.

Comparación visual de regularizadores



Weight decay atrae hacia cero; soft sharing atrae hacia centros aprendidos.

Weight decay $\Rightarrow$ pesos pequeños,
Priors consistentes $\Rightarrow$ regularización por grupos,
Early stopping $\Rightarrow$ capacidad efectiva limitada,
Tangent propagation $\Rightarrow$ invariancia local,
Data augmentation $\Rightarrow$ invariancia mediante ejemplos,
CNNs $\Rightarrow$ restricción estructural,
Soft sharing $\Rightarrow$ pesos agrupados suavemente.

Regularizar puede significar

preferir funciones suaves, preferir invariancias,
preferir estructuras compartidas, preferir modelos que generalicen.

Idea principal

La regularización no solo controla parámetros: incorpora conocimiento previo sobre qué soluciones son razonables.